

專案開發紀錄：個人伺服器架構（Homelab）

背景資訊

前陣子，我朋友常炫耀自己的自架 NAS，但因當時對於「自架需要高級的硬體」與「要另外購買硬體才能自架」的誤解而打消了念頭。直到有一次出門想分享手機裡的照片，才發現忘了從電腦同步過來，又不想把所有照片交給 Google Photos 管理。這時我意識到，像照片備份這種需求，用手邊現有的筆電就能自己架設。

架起 Immich（照片備份）之後就停不下來了：Navidrome 讓我擁有自己的音樂串流；ConvertX 讓我不用等線上轉檔網站的廣告跑完就能批次處理檔案；AdGuard Home 讓全家的裝置都不再看到廣告。一項一項疊上去，最終變成了十餘個 Docker 容器同時運行的個人伺服器。

實作內容

在 Arch Linux 上以 docker compose 容器化部署所有服務，並透過以下架構實現外部存取與安全防護：

- 反向代理：Caddy（含 Cloudflare DNS Plugin），統一管理所有子網域的 HTTPS 憑證自動簽發與路由。
- 公網穿透：家中網路位於 CGNAT 之後沒有公網 IP，因此使用 Cloudflare Tunnel（cloudflared）以 host network 模式運行，將「子網域.7279744.xyz」的流量導入本地 Caddy。
- 內網互通：Tailscale VPN 建立跨裝置的私有網路，搭配 AdGuard Home 的 Split DNS 將內部請求直接指向 Tailscale IP，避免繞行公網。
- 安全防護：iptables 手動設定 INPUT DROP 策略，僅開放 Tailscale 與必要端口；CrowdSec IPS 分析 Caddy access log 自動封鎖異常 IP。
- 遠端 SSH：透過 Cloudflare Access Zero Trust 提供瀏覽器 SSH（ssh.7279744.xyz）並且另外架設 GitHub OAuth 驗證

目前執行的服務

AdGuard Home（DNS）、Navidrome（音樂）、Immich（照片）、Dockge（Docker 管理）、ConvertX（檔案轉換）、2FAuth（雙因素驗證）等。

反思

架設過程中最大的收穫來自於工具的替換與踩坑。一開始我照朋友的建議使用 **Nginx Proxy Manager** 的圖形介面管理反向代理，但在發現 **Caddy** 能自動從 **Let's Encrypt** 取得 **HTTPS** 憑證後決定轉換。過程並不順利：**Caddyfile** 的巢狀區塊語法、**Cloudflare Tunnel** 的整合方式、**redirect** 規則的優先順序，每一步都跟我的直覺不同，花了大量時間反覆查閱官方文件與社群討論才逐步解決。